第 11 章　全自动加法计算机

能够将一大堆数加起来，这确实很不错，麻烦的是要来来回回地拨弄一大堆开关，恐怕不能拿它到市场上买菜。而且，要想用好它，得先上培训班学二进制。

买菜的事情先放一放，不要忘了我们原先的目标——从头制造一台现代的计算机，在不需要人工干预的情况下自动计算。当然，要达到这个目标还需要一段距离，也许是 50 米，看起来不太遥远，但终究还是需要一个过程。现在的问题是，要把一大堆数加起来，靠手工操作不是很方便，而且说不定什么时候一个不注意，把开关扳错了，你还得从头再来一遍。要是有几百上千个数相加，而这个错误恰恰发生在只要加完最后一个数就可以完事大吉的时候，你一定会拿开关砸自己的脑袋。

为了避免你的脑袋受伤，能不能把所有的数提前存起来，然后再让一台机器自己一个一个地取出来相加呢？尤其是考虑到这样做还有一个特别的好处，那就是如果有些数搞错了，可以单独修改，然后让机器从头再算一遍，反正它是自动的。能不能发明出这样一种计算机器呢？

这当然难不倒以聪明著称的我们。通常，一个能保存很多二进制数的东西叫做存储器。

11.1　咸鸭蛋坛子和存储器

提起存储器，我们马上会想到袋子、盒子、坛子、罐子、箱子和柜子。实际上，这些东西的确是存储器，但不是我们想要的那种。我们需要的是能保存二进制数的存储器。

尽管保存的内容不一样，但所有的存储器都有一个共同特点，那就是它们通常都只有一个口。通过这个口，可以把东西放进去，或者把里面的东西拿出来。想想看，要是每个储存在坛子里的咸鸭蛋都有自己的出入口，这坛子也就成了大喷壶。基于同样的原因，一个能保存许多二进制数的存储器也应当只有一个出入口。你可以把它想象成一个大箱子，里面装了许多二进制数，但这个大箱子只有一个口，一次只能存入或者取出一个。

最小的——我指的是存储容量——存储器就是我们熟悉的上升沿 D 触发器，它可以，而且仅仅只能保存 1 个比特。

我当然很清楚，存储器最好是用来保存一大堆二进制数，而不是一大堆单个的比特，就像你的书柜应该用来存放大量的书，而不是堆满了没有装订在一起的纸。但是，存储单个比特的触发器是制造大容量存储器的基础，要是你有能力保存一个比特，我们才有理由相信，你同样可以保存更多的比特。

考虑到存储器和咸鸭蛋坛子的相似性，它应该只有一个出入口。不过，论制作工艺，前者比后者烦琐，因为它必须使用传输门。如图 11.1 所示，这个存储器由一个上升沿 D 触发器构成，只能保存一个比特。

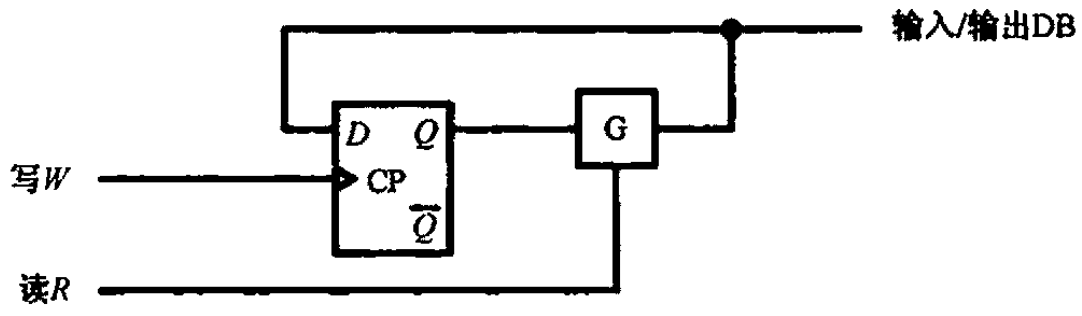


图 11.1　具有唯一输入/输出线的存储器，它可以保存或者读出一个比特

对于 D 触发器没有什么可说的。G 是我们的老朋友传输门，这位仁兄的作用是将 D 触发器的输出 Q 同外部接通或断开。

和往常不一样，对于存储器来说，工程师们已经习惯将保存一个数称为“写”（Write）；而从存储器里取出一个数则称为“读”（Read）。不管是“读”还是“写”，一律称为“访问”（Access）[①]。所以，“W”和“R”分别用于从这个存储器中写入或者读出一个比特。

平时，W 和 R 都为 0，这个存储器什么也不做，既不能写入，也读不出比特，因为传输门 G 是断开的，而触发器的 CP 端也没有接到任何有效的指示。

这是存储器的默认工作状态，或者说是它一贯的生活方式。不过，要是你有一个比特想托付给它，它也不会让你失望。当然，前提是你要在 DB 上准备好这个比特，然后使 W 从平时的 0 变为 1（上升沿）。如图 11.2 所示，在 W 脉冲的上升沿，比特被 D 触发器保存。替你保管了这么重要的东西，挺辛苦的，你应该让人家恢复以前的平静生活。所以，在此之后，你应当使 W 重新为 0。

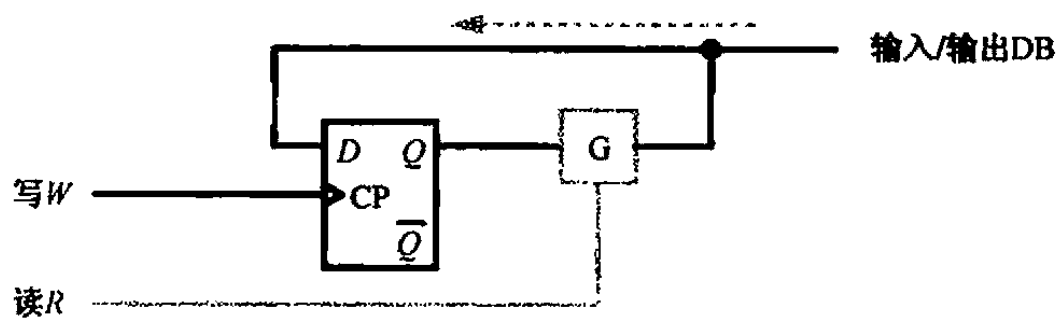


图 11.2　写入一个比特的原理

① 单从这些术语上看，机器的生活和我们人类差不多，唯一不同的是我最喜欢清静，而你无论怎样访问机器，它都不会眉头紧皱、怨气冲天。

写入比特的时候，应当使 R 保持它平时为 0 的状态，毕竟你要“写”而不是“读”。在这种情况下，传输门 G 是断开的，来自 DB 的输入和 Q 上的输出不会遇到一起。这是好事情，否则，一定会发生一场冲突，甚至起烟冒火。这种场面虽然很好看，但绝对不是你想要的。

相反地，如果是要读出数据，那么如图 11.3 所示，必须使 $R = 1$ 以打开传输门，触发器 Q 端的比特被送入 DB 总线。

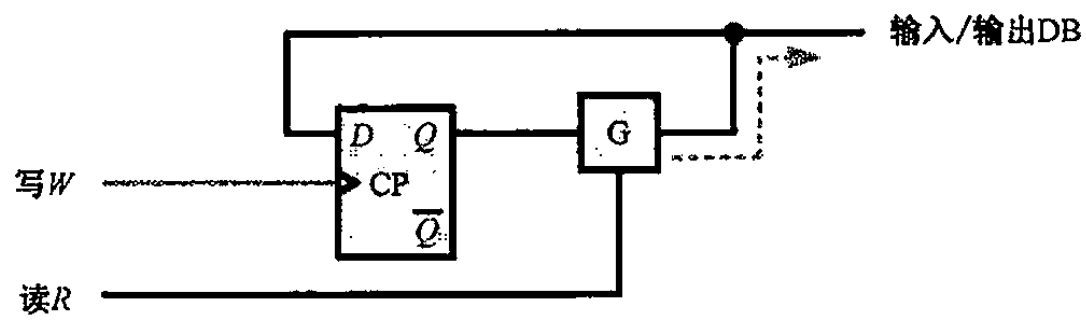


图 11.3　读出一个比特的原理

没错儿，这个输出 Q 同时也被送入触发器自己的输入端 D。但是没有关系，就让它来吧，因为在读的过程中，W 必须为 0。最后，如果你确信外部已经将这个比特取走，则必须使 $R=0$ 以打发存储器继续过清静无为的日子。

我们已经暗示过，W 和 R 不能同时从 0 变为 1，因为这是很奇怪、很无理的要求，意思是我既想读又想写。要是你非这样做，那么存储器会做一个很奇怪的动作：吞食自己的输出，这是没有任何意义的。

只能保存一个比特，这个存储器的容量未免太小了些。不过，要是你能把一棵树栽活，那么就可以用同样的方法栽种更多的树，从而形成一大片森林。这意味着，它可以作为一个基本单元，来制造更大容量的存储器。出于这个原因，我们姑且称之为比特单元。为了方便后面的讲解，我们给它画一幅肖像，如图 11.4 所示。

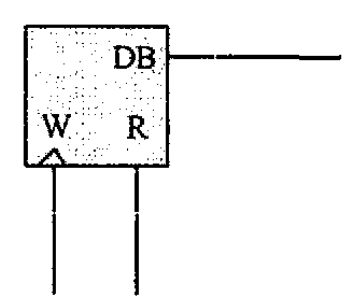


图 11.4　比特单元示意图

毕竟是和电路有关的东西，所以它不如张择端的《清明上河图》来得大，也不如它有趣。再加上刚刚学过它的工作原理，我也不想再多说什么。如果非要说些什么，还是那句老话，只读/写 1 个比特几乎没有什么用处，我们制造存储器，是希望它能保存完整的二进制数。

这个愿望很难实现吗？不是的。如图 11.5 所示，因为一个二进制数通常包含许多个比特，是一个比特串，所以，你可以把很多比特单元并排组织起来，以容纳该二进制数的每一位。

使用多少个这样的比特单元，取决于你要保存的二进制数有多大，换句话说，它包含了多少个比特。在这个例子中，我们使用了 5 个比特单元，所以它只能保

存像 11010 这样的二进制数。

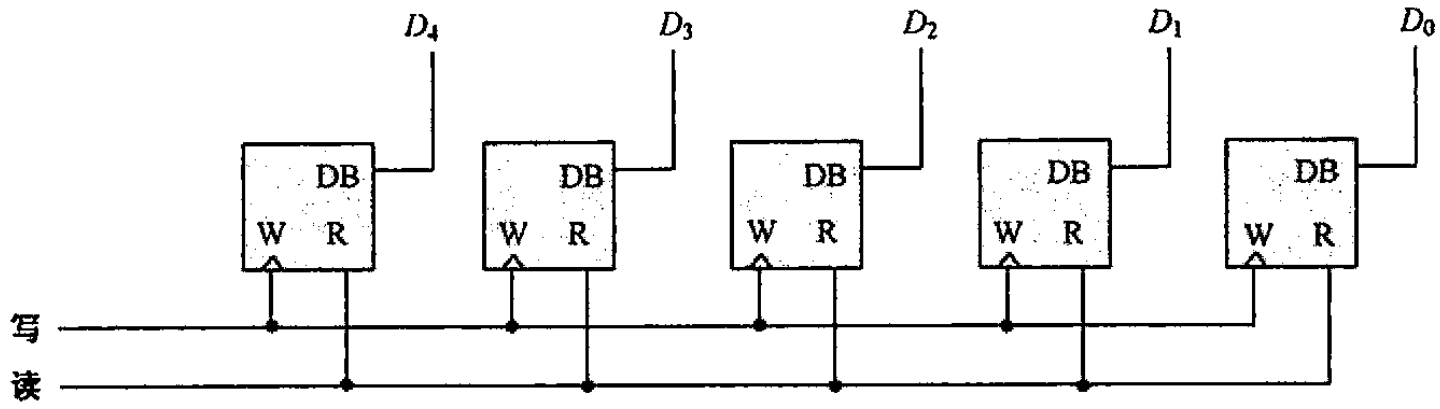


图 11.5　可以读/写单个 5 位二进制数的存储器

这样，当你要写一个二进制数时，可以把它的每一位分别放到 D_0～D_4这 5 根线上，保持“读”线为 0 不变，并通过“写”线发出一个上升沿脉冲，这时，它们将分别被独立地保存起来；相反，当你要读出这个二进制数时，只需要使“写”线保持为 0 不变，“读”线为 1 即可，它的每一位会自动出现在 D_0～D_4上。

直观上来说，这 5 个比特单元并排在一起，很像一个楼层。既然一层楼可以保存一个二进制数，那么，只需要多盖几层，就可以保存好几个二进制数，不是吗？如图 11.6 所示，我们一共盖了 4 层，可以保存 4 个二进制数。当然，要是你想保存更多，完全可以盖更多层。

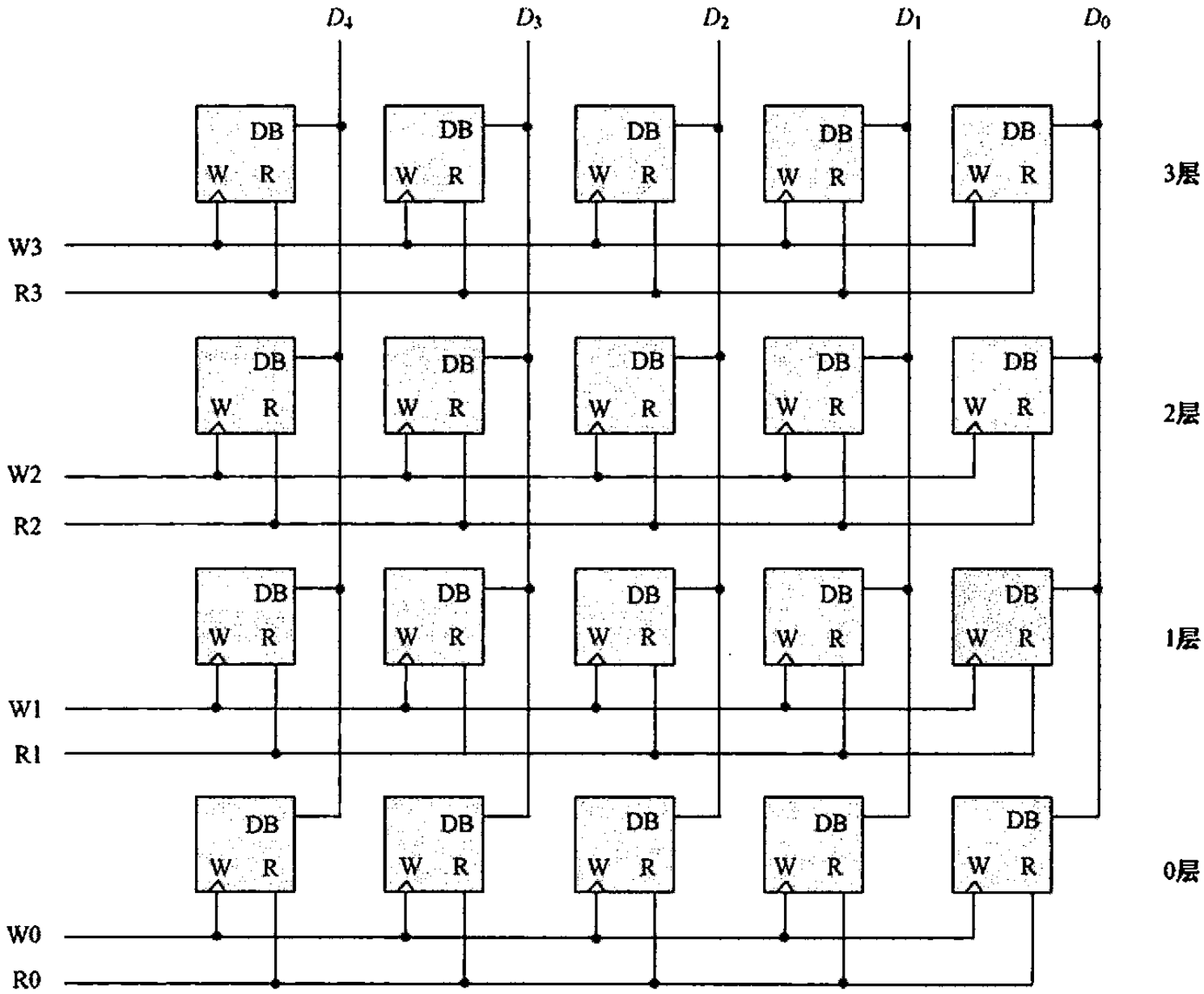


图 11.6　存储器示例，它可以保存 4 个二进制数、每个数包含 5 个比特

既然我们把它看做楼层，那么，为了指明把一个二进制数保存到第几层，或者从第几层读出来，需要给每一层编号。不过，和平时不同，我们不是从一楼开始编号，而是要从地下室开始，所以，这4个楼层按顺序应该是0、1、2、3层。

每个楼层都有自己的读线和写线。比如，0层的读线是R0，写线是W0；第1层的读线是R1，写线是W1，其他各层依次类推。平时，每一层的读线和写线都为0，它们既写不进去，也读不出来。要是你想往这个存储器里写入一个二进制数，只需要使相应楼层的写线从0翻转到1即可，读的时候也与此类似。当任何一个楼层正在读/写的时候，其他楼层都处于休眠状态，既不能读，也不能写，所以它们决不会互相干扰，这一点大可放心。

为了操纵读线和写线，给它们通电，或者使它们断电，我们这些业余选手只能在家里使用传统的开关。问题是，少量的楼层还好办，要是这里有成千上万的楼层，你就得拿成千上万的开关，找成千上万的电线。这可真是宏伟的工程，这么多的开关，对于任何一个五金商店来说都是大买卖。第二天，你光顾过的商店都将关门歇业，原因是他们再也没有开关可卖，都高高兴兴地出去进货了。

除了体积和数量上的因素外，想想看，要是你准备往99999楼层写入一个二进制数，还得抹着汗找那个第99999号开关W99999，这会是一种什么场景？这开关就不能省省吗？

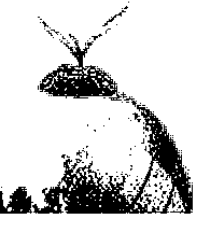
怎么说我们也是这个星球上最聪明的物种，自然也能想出好办法来。每个楼层不是都有编号么？我们可以通过指定编号的方法来告诉存储器，我们要访问的是哪个楼层。

通过编号来指定存储单元不是用嘴说，因为这些家伙听不懂人话。所以这需要一排开关，通过扳动开关来拼成二进制数，每个二进制数都表示一个楼层的编号。这是非常巧妙的方法，假如我们有15个开关，就可以拼出任何一个15位的二进制数，从000000000000000到111111111111111，也就是十进制的0到32 767。换句话说，只需要15个开关就可以指定32 768个楼层中的任何一个，从0号到32 767号，而不是像以前那样，为每个楼层都配备两个开关。

32 768个楼层，对于这本书来说太高了，画不下，所以还是来看前面那个例子。它只有4个楼层，编号为0、1、2、3，分别对应二进制数00、01、10、11，都是2比特的二进制数，正好需要两个开关就足够了。

话虽这么说，但是要想用两个开关来从4个楼层中选出一个，实际上该如何做呢？你要如何设计电路连接呢？当然，答案是发明一个新逻辑电路，用来选择一个楼层。

如图11.7所示，你也看到了，这个新的逻辑电路叫做“地址译码器”。这个名字是啥意思，我们稍后再说，重要的是你要知道，A0和A1共同用来指定一个楼层。如果A1A0＝00，表示它选择的是第0层；如果等于11，则指的是第3层。



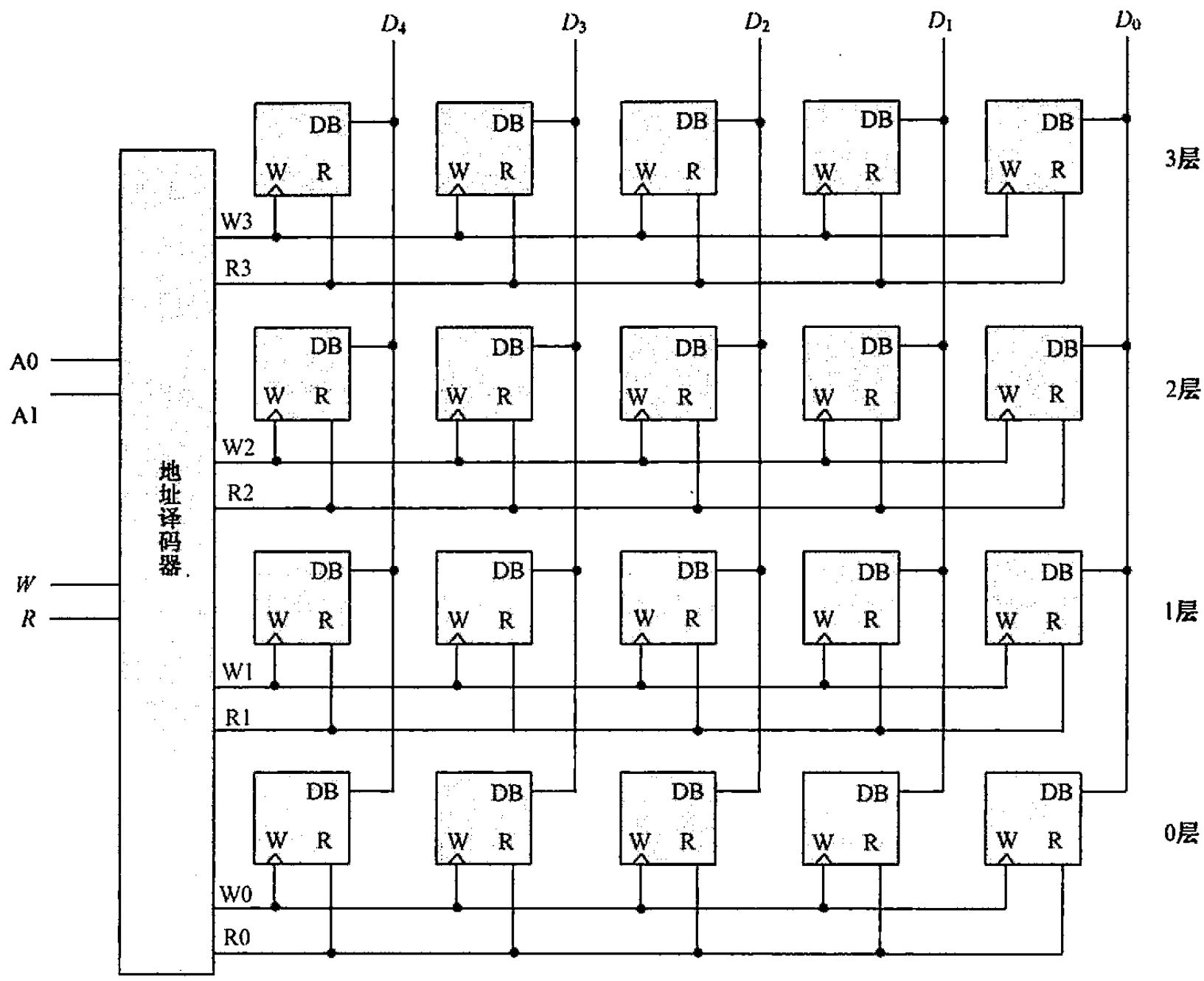


图 11.7　地址译码器对于简化存储器设计是必不可少的

该逻辑电路还有另外两个输入，也就是 *R* 和 *W*，它们用于指明对所选择的楼层进行何种操作，是读还是写。要是 *R* 和 *W* 都为 0，那么，不管你选择了哪个楼层都没有用，因为此时 *R*0、*W*0、*R*1、*W*1、*R*2、*W*2、*R*3、*W*3 都为 0。

但是，假如你选择了第 2 个楼层，即 A1A0＝10，那么，当 *W*＝1 时，W2＝1；相反的，当 *R*＝1 时，R2＝1，这就是该逻辑电路的工作原理。

对于存储器来说，不管是楼层也好，编号也罢，这都是打比方。要想进入计算机行业，和那些严肃的前辈们做朋友，你必须改改口，不要再楼层编号什么的，要说“地址”。这实际上是借用了生活中的词汇，好处是容易理解和接受。没有地址，你将收不到远方朋友寄来的新春贺礼，而通过指定一个地址，就可以从这个存储器的某个位置取出一个数字，或者写入一个数字。

地址仅仅是一个编号，一个门牌号码，它指向了存储器内部的一个小空间，这是真正用于保存数据的地方，这个地方就叫存储单元。毫无疑问，每个地址都对应着一个存储单元。

一旦通过 A0 和 A1 给出一个存储单元的地址，那么，结合 *R* 和 *W* 的输入情况，就能得到另外一种形式的输出，使相应的存储单元开始读或者写，这实际上就是一个转换或者翻译的过程，这就是为什么我们称之为“地址译码器”的原因。

构造地址译码器需要详细地定义一张符合其工作状态的真值表，并依据该表写出各项输出的逻辑表达式。在这本书里，这样的工作已经做了不止一次，再做一次已经没有必要。尽管他们嘴上没有说，但我想出版社的编辑肯定不希望这本书太厚，所以这就当是留给你们的作业吧。

俗话说“人靠衣服马靠鞍”，凡事都讲究个包装。金玉其外，败絮其中，这包装的坏处，很多人已经在网络上、书本里说过了。不过，包装也不全是坏处，你看看这个复杂的存储器，要是把它包一包、装一装，放进盒子里，就既好看，用起来肯定也特别方便。

如图 11.8 所示，这是封装之后的存储器。它有 4 个地址引线 A0～A3，可以访问 0000～1111 这 16 个存储单元，这就是它的存储容量。另外，它有 D_0～D_4 五根数据线，这意味着它每次可以写入或者读出一个 5 比特的二进制数。

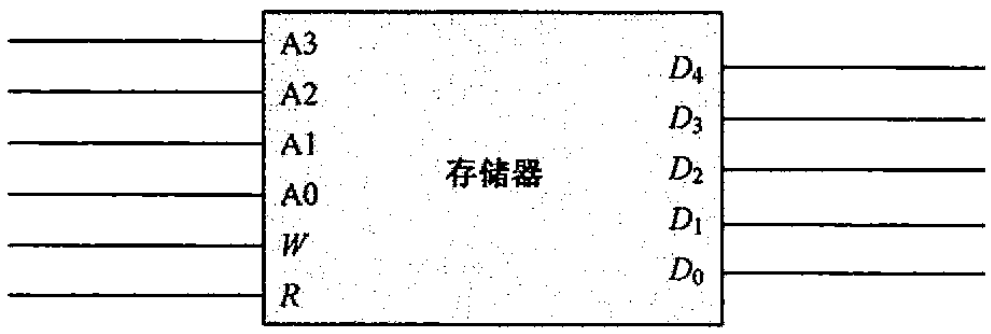


图 11.8　封装后的存储器整体外观

在生活中，多数的存储器，比如咸鸭蛋坛子，都只能按顺序一点一点地存放或者取出，上面的蛋不拿走，下面的拿不出来。但是我们这个存储器，你可以随机地、任意地决定访问哪个存储单元，不管访问哪个存储单元，所花的时间都一样，和地址没有关系。正是因为这样，我们通常称之为“随机访问存储器”，或者“自由存取存储器”，用英文来说就是 Random Access Memory，简称 RAM。由于组成它的细胞是触发器，而这种东西就怕断电，不管它记下的是什么，只要一断电就全完蛋了，因此属于易失性存储器。

11.2　磁芯存储器

用触发器来制造存储器，这似乎是很轻松、很容易的事情。而且从理论上来说，无论多大的存储器，都可以用触发器堆叠出来，即使你有成千上万，甚至几十万、几百万个二进制数需要保存。

纸上谈兵很容易，但是千万别玩儿真的。要是存储容量很小很小，采用触发

器来制造倒还是可以理解的，当存储容量变得很大时，这种搞法就行不通了。

一般的，保存1个比特的成本是好几个电子管或者晶体管。假如需要5个（实际上这根本不够），那么，按一个二进制数有8个比特来算，要保存100个二进制数至少需要5×8×100=4 000个，这还没把地址译码器算在内。

老实说，一个存储器只能保存100个二进制数，搁现在就是扔到垃圾堆里都没人要。但在电子管和晶体管的时代，却能让科学家们在梦里笑醒。你想想，好几千个晶体管，连线有多复杂，体积有多大，需要费多少电？当然，还包括钱，也就是科研经费。20世纪30年代末，也就是电子管发明30年之后，一只性能好一点的电子管需要花费10美元才能买到。而那个时候，正是计算机开始跑步前进、需要大容量存储器来为它提供原料的时候。

困难很多，而存储器也是需要的。没办法，人们只能绞尽脑汁，想了很多古怪的方法来制造存储器。在这些曾经用过的东西当中，占统治地位的是一种叫做磁芯的物什。

我们都知道电磁学，也明白电和磁的关系。对于像钢这样的东西，容易被磁化，即使本来没有磁性，但在和吸铁石接触之后也会变成一块磁铁，这叫做剩磁。和钢一样，磁芯用铁氧体材料制成，像一个圆环，外直径通常在0.2～2mm之间，比一粒芝麻大不了多少。

用磁芯来保存数据是利用了电流能产生磁场，有些东西在磁化之后会产生剩磁，同时，磁场反过来也可以产生电流的原理。而且更重要的是，不同的电流方向将使磁芯按不同的方向磁化，换句话说，电流从左边流向右边，和从右边流向左边，这两种情况下磁芯的南北极是截然相反的。这也意味着，如果把一种剩磁状态看成是“0”而另一种状态看成是“1”，那么磁芯可以用来保存1个二进制比特。

如图11.9所示，在磁芯中穿一根电线，叫做驱动线，用来往磁芯中写入一个比特，所以称之为写入线也是可以的，但最好还是称做驱动线，因为读出比特的时候也要用到它。要达到写入一个比特的目的，需要控制电流的方向。

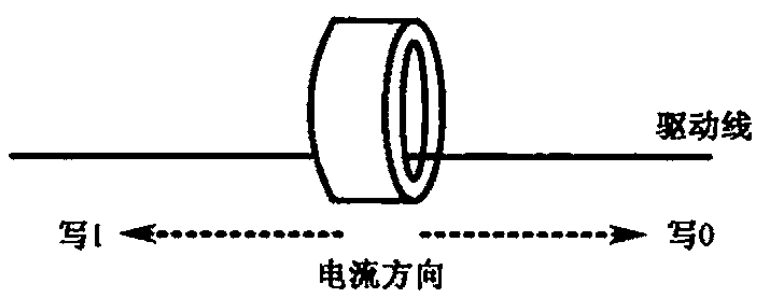


图11.9　电流方向决定了磁芯的磁化方向

往磁芯中写入一个比特是比较简单明了的，但要把这个比特读出来就很古怪，需要在它里面穿另一根电线，叫做读出线，如图11.10所示。现在，你面临着和

法拉第一样的困惑：只是在磁场中穿一根电线，这毫无用处，静止的导体不会产生电流，如何在读出线中感应出这个比特数据呢？

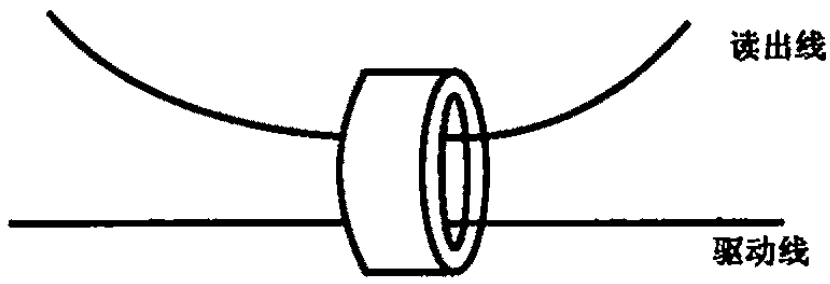


图 11.10　磁芯的工作原理

读出比特的过程要麻烦一些，但是很有趣，方法是用驱动线向磁芯中写入一个比特“0”。如果磁芯中保存的本来就是个“0”，那么这个写入电流不会对原有的磁场产生太大的影响，以至于读出线上感应出的电压很小，这表示读出的是“0”。但是，如果磁芯中原来保存的是“1”，那么这个写入电流将使磁芯的磁场翻转，这种变化即使算不上翻江倒海，也非常强烈，这个大幅度变化的磁场将在读出线上感应出较高的电压，这表示读出的是“1”。

显然，磁芯的读出是破坏性的，如果读出来的是“1”，那么读取之后磁芯的状态将会是“0”，这需要重新将读出来的“1”写回磁芯中，这是磁芯存储器比较麻烦的地方。

从磁芯中读出的电压很小，而且不是我们需要的方波。好在有电子管和晶体管，可以将其放大并进行规整，使之合乎要求。和触发器不同，磁芯有一个明显的好处，那就是断电之后，也能够维持写入的数据。

磁芯在全世界应用了好几十年，被实践证明是那个时代最好的存储器制造材料，发明它的人是美国华裔科学家王安博士。王安祖籍江苏昆山，1920 年出生于上海，先后就读于上海交通大学、哈佛大学，1948 年获得哈佛大学博士学位。

1949 年 10 月 21 日，王安申请了磁芯存储器的专利，几年之后，又创办了自己的公司，生产小型商用计算机和文字处理机，事业兴旺，盛极一时。1988 年，王安被列入美国发明家名人堂，自 1901 年创建以来，只有爱迪生等 68 人入选名人堂。非常遗憾的是，从 20 世纪 80 年代后期开始，王安的公司迅速衰落，最后销声匿迹，令人扼腕。

中国的第一台电子计算机用的也是磁芯存储器。在当时，研制计算机是高度机密的事情，所以穿磁芯这活儿不能交给外边的人来干，只能在计算所内部消化。据说有一个女研究员干起这活儿来总是比别人快好几倍，怎么追也赶不上，直到过春节表演节目的时候才算真相大白，原来人家以前是部队文工团表演魔术的，吃的就是眼疾手快这碗饭。

11.3 先存储，后计算

但愿你还没有忘了为什么我们要发明存储器。现在让我们在第10章的基础上继续发明创造，来搞清楚如何让机器自动取数，然后计算。

因为我们是要计算

$$10+5+7+2+6$$

而且，我们还想预先把这些要加的数都写入存储器，然后再一个一个取出来相加。为此，我们可以使用图11.8那样的存储器。一方面，这5个数都不大，每一个的长度都不超过5个比特；另一方面，要加起来的数只有5个，而这个存储器却有16个存储单元，空间是足够了，而且还绰绰有余。

在开始做加法之前，先要把上面那5个要加起来的数写到这个存储器里。如果没有特殊的原因，所有的二进制数都应该从存储器的顶端，也就是地址0000开始一个挨着一个存放，如图11.11所示。

地址	存储内容	十进制
0000	01010	10
0001	00101	5
0010	00111	7
0011	00010	2
0100	00110	6
0101	未用	
⋮	⋮	
1111	未用	

图11.11 将所有要加起来的数顺序存放在连续的存储单元里

为此，你可能要用4个开关来形成地址，再用5个开关拼成要写入的二进制数，最后一个开关连在存储器的W端，用来向存储器下达写入命令。从地址0000开始，每当一个地址和一个二进制数准备好后，按一下W开关，就这样操作，直到把所有的数都按顺序写入存储器。在这个过程中难免会出点儿小差错，但是存储器的好处就是你可以反复修改任意一个地址里的内容。

因为我们只是把5个数相加，所以前5个地址，从0000到0100，里面的内容是我们特意写入的。其他地址，也就是从0101到1111，没用使用。

正常情况下，要把刚才写进去的数一个一个地读出来，同样需要给出地址，然后使R=1。不过我们的想法有些古怪，希望能够用最省事的方法连续操作，把它们按顺序一个一个地取出来，毕竟目标是将它们按顺序相加。当然，最好是像第10章那样，拍拍开关就能做到。

其实不太难，图11.12就是我们给出的方案。

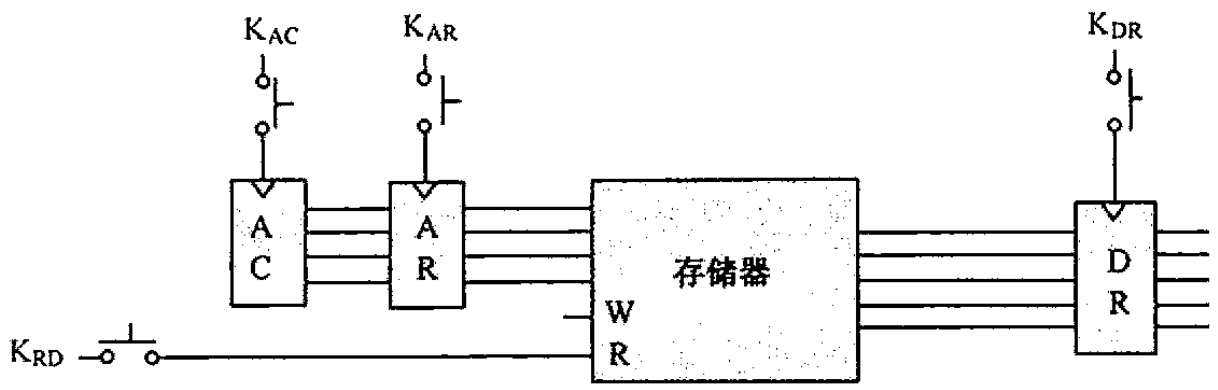


图 11.12　顺序地从存储器里取数的电路方案

如图中所示，假设存储器里已经存放了我们要加起来的 5 个二进制数；AC 是计数器，用以提供访问存储器的地址，所以称为地址计数器。一开始它的内容是 0000，每按一次 K_{AC}，它就在原来的基础上自动加一，以得到访问下一个存储单元所需要的地址；AR 是一个寄存器，用来临时存放存储器地址，称为地址寄存器。看起来 AR 有些多余，似乎用 AC 给存储器提供地址更直接。但是你很快就会发现，不把计数器 AC 直接和存储器相连，而是由 AR 负责转交是非常有道理的。

很显明，K_{RD} 的作用是给存储器发出命令，要求它将数据送出。注意存储器的 W 端没有使用，因为我们现在只是要读，所以将它悬空，让它一直为 0。

在存储器的数据端，数据寄存器 DR 用于暂存读出的数据，除了名字上的不同之外，它和普通的寄存器没有什么两样。存储器有自己的事情，它只负责把客人送到门外。所以数据应当在 DR 中稍事休息，等待进一步的指示，以决定自己应该动身前往何处。

在介绍完这一家人之后，现在可以接二连三地往外取数了。在开始之前，先想办法把地址计数器 AC 清零，以指向地址 0000，然后执行以下操作：

（1）按一下 K_{AR}，地址计数器 AC 当前的值 0000 被 AR 锁存，并提供给存储器。

（2）先按住 K_{RD}，不要松开，再按一下 K_{DR}；这时，数据送出，并被 DR 保存；最后，松开 K_{RD}；

（3）按一下 K_{AC}，地址计数器加一以指向下一个地址，为再次从存储器里读数据做准备。

至此，存储器里的第一个数就被取出来了。如果要接着取第二个数、第三个数，等等，重复按上面的三个步骤操作即可。

在第 10 章里，我们已经学会如何只用一个按键开关、一个循环移位寄存器和一个逻辑电路简单地操控、把一大堆数加起来。道理都是一样的，按照这种思路，我们同样可以发明出一个控制器，用它来简化取数过程，如图 11.13 所示。

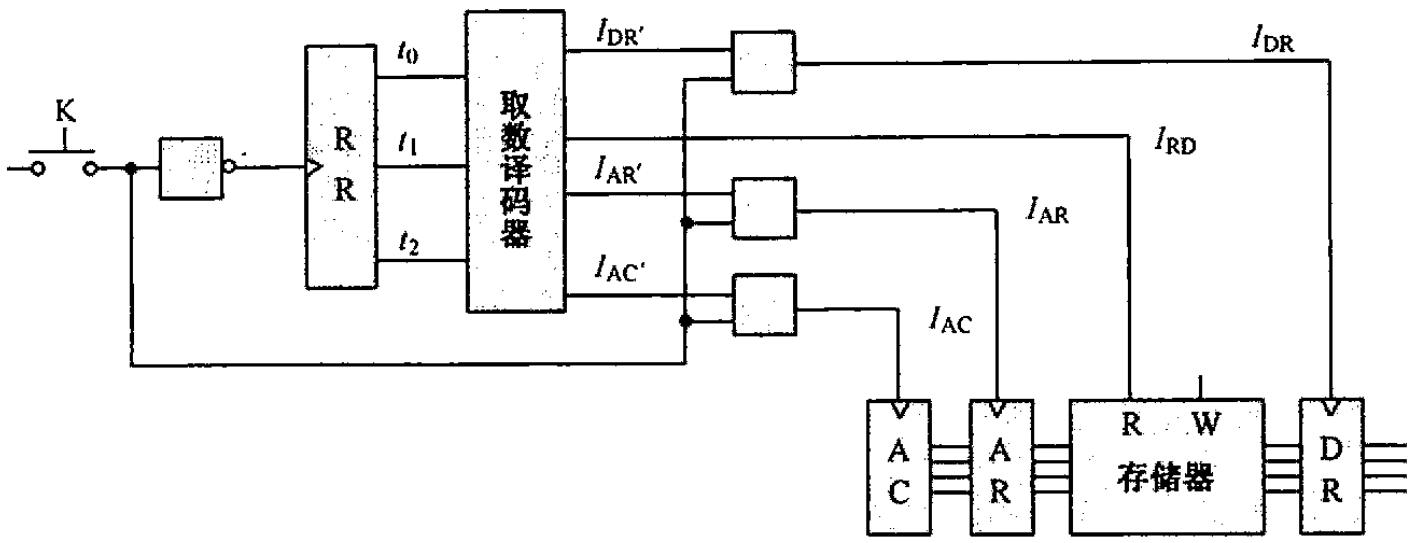


图 11.13　用一只开关依次将数取出

图中，RR 就是我们所说的循环移位寄存器。和第 10 章的那个不同，它有 3 个输出 t_0、t_1 和 t_2，毕竟我们刚才已经看到了，把一个数从存储器里取出来，需要经历三个不同的步骤。

同样是在第 10 章里，为了制造控制器，我们发明了一个逻辑电路。因为不知道该给它起个什么名字，只好叫做“我们的新电路”。现在看来，它其实就是一个译码器，把一种形式的输入转换翻译成另外一种形式的输出。在我们现在的这个控制电路里，同样需要一种类似的译码器，因为发明它的目的是从存储器里取数，所以称之为“取数译码器”。

电路刚刚启动的时候，$t_0=1$ 而 t_1 和 t_2 都为 0，此时，只有 $I_{AR'}=1$，如图 11.14 所示。

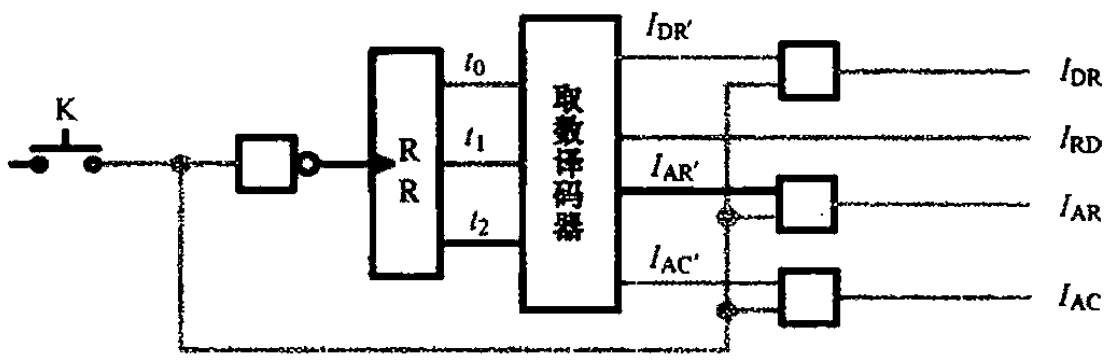


图 11.14　取数控制器的原理（1）

一旦开关 K 按下，则 $I_{AR'}$ 和从 K 来的“1”一起，通过与门使得 I_{AR} 从 0 翻转到 1，于是地址寄存器 AR 将地址计数器的地址保存，并提供给存储器。对于循环移位寄存器 RR 来说，从非门得到的是一个下降沿，它所能做的，就是无动于衷，不予理睬，如图 11.15 所示。

当 K 松开时，RR 得到一个上升沿脉冲，于是 $t_0=0$，$t_1=1$，$t_2=0$，并因此使得 $I_{RD}=I_{DR'}=1$，如图 11.16 所示。此时，存储器开始向外送出数据，但还不能被 DR 寄存器保存。

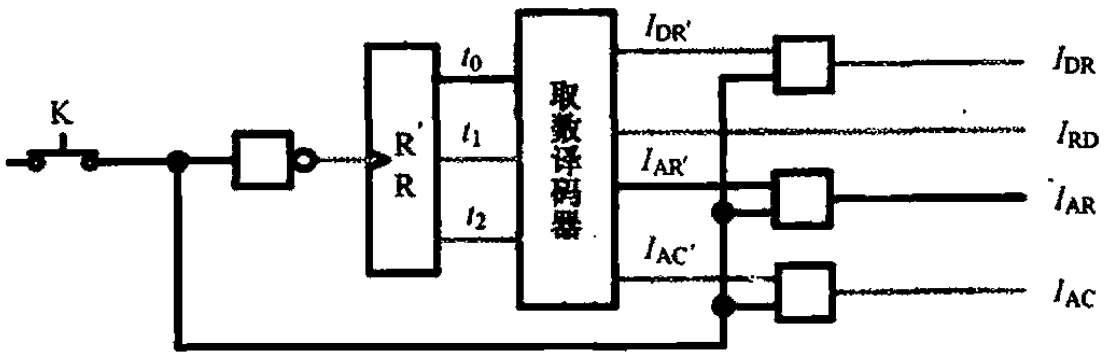


图 11.15　取数控制器的原理（2）

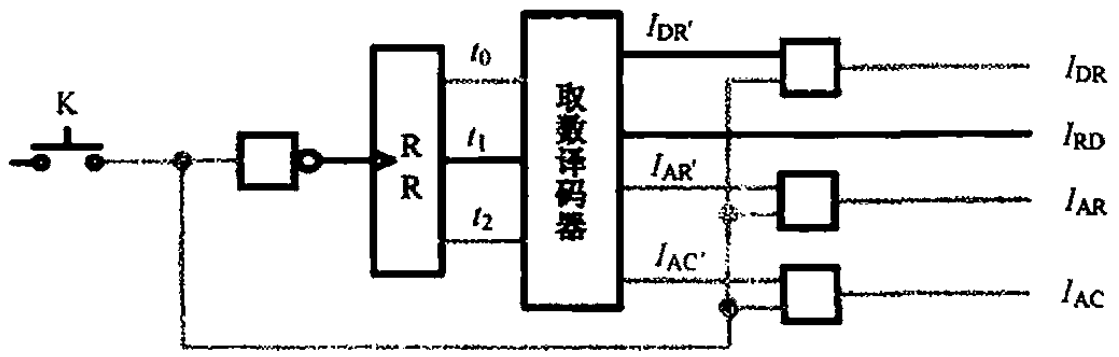


图 11.16　取数控制器的原理（3）

当 K 第二次按下时，RR 依然无动于衷，但是 I_{DR} 却立即从原来的 0 翻转到 1，于是寄存器 DR 将存储器送出的数字保存起来，如图 11.17 所示。

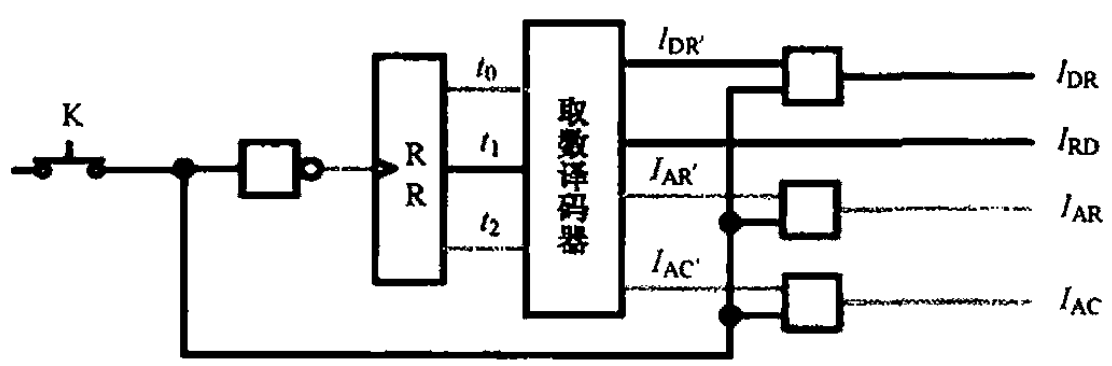


图 11.17　取数控制器的原理（4）

K 再次松开后，$t_0 = t_1 = 0$ 而 $t_2 = 1$，于是 $I_{AC'} = 1$，如图 11.18 所示。

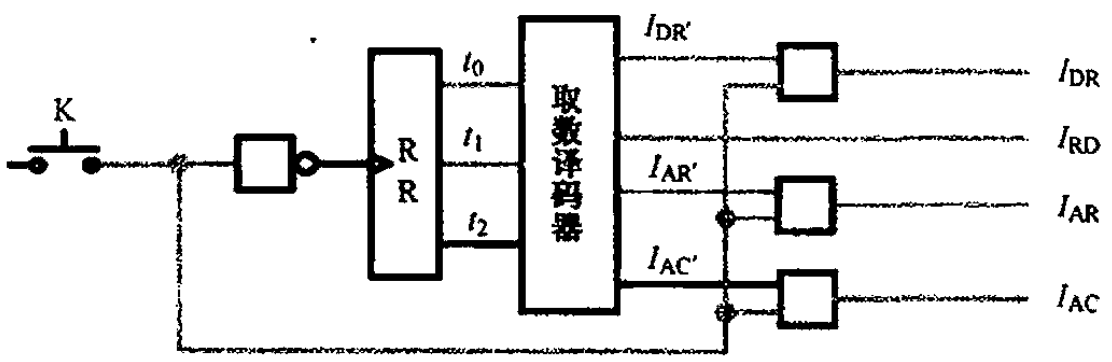


图 11.18　取数控制器的原理（5）

现在，我们第三次将 K 按下，这将使得 I_{AC} 从 0 翻转到 1，于是地址计数器自动加一，以指向下一个存储单元的地址，如图 11.19 所示。

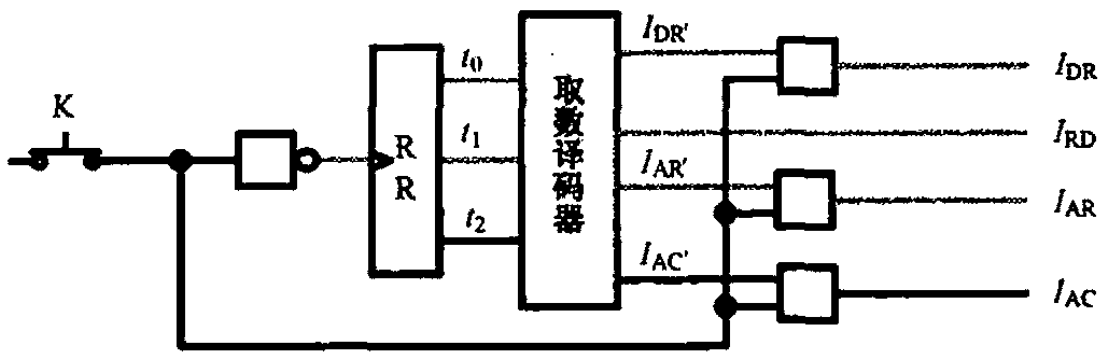


图 11.19　取数控制器的原理（6）

一旦 K 第三次松开，RR 将再次循环移位一次，使得 $t_0=1$ 而 $t_1=t_2=0$，这又回到了一开始，即图 11.14。换句话说，如果再连续按三次 K，将会把下一个存储单元里的数取出来。

11.4　半自动操作

我们的目标是用机器计算一连串的加法，比如：

$$10+5+7+2+6$$

还记得为什么要发明存储器吗？那是因为我们有一个美好的愿望——把所有要相加的数都提前保存起来，然后，我们什么也不干，就坐在那里看机器自动地把它们取出来相加。目标就在眼前，看来只有 20 米，让我们继续前进。

在第 10 章里，所有参与相加的数都是用开关得到的。现在，存储器和加法器可以合在一起，以实现从存储器里不断取数、然后相加的功能，如图 11.20 所示。

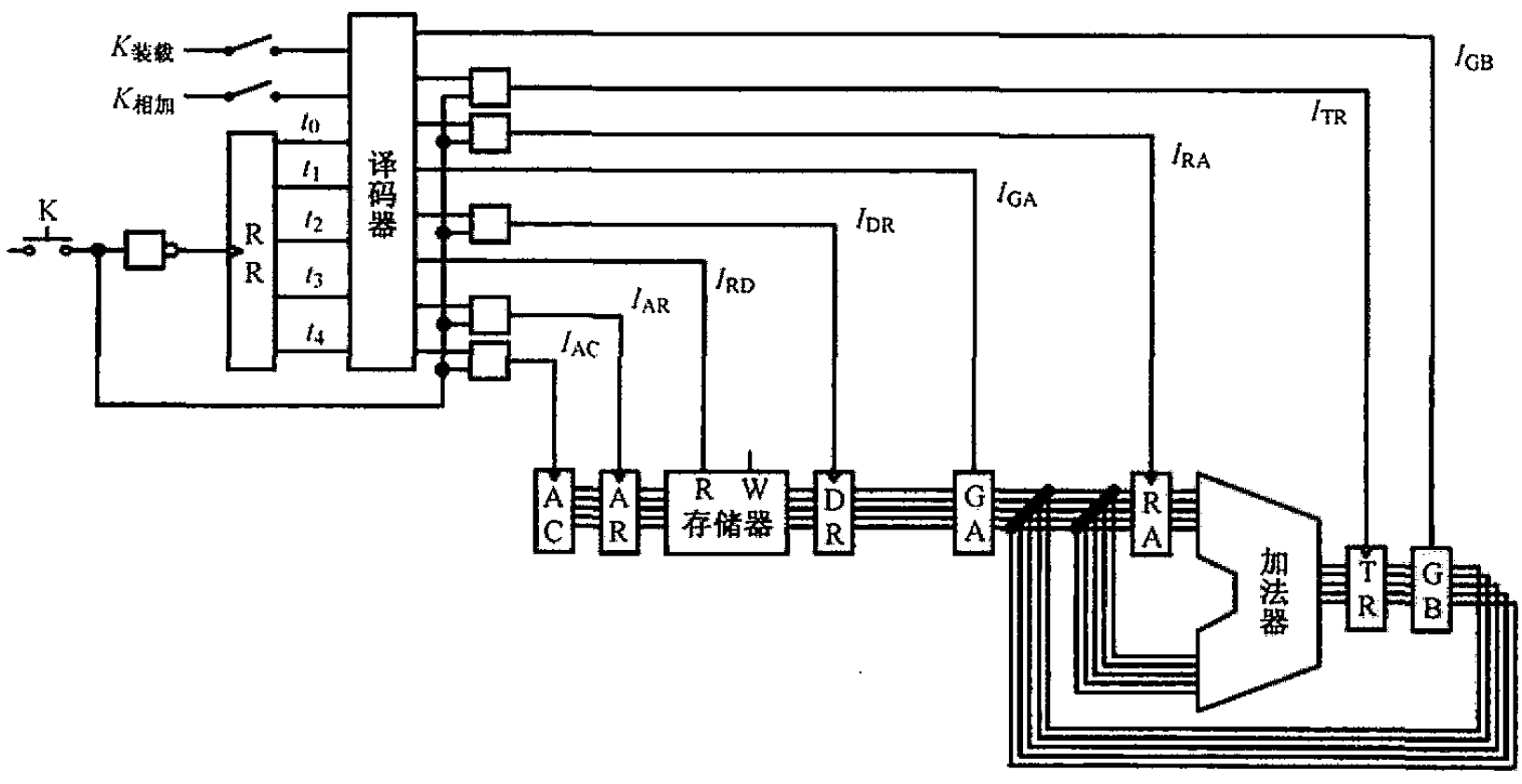


图 11.20　部分实现了自动化的连续加法电路

新的控制电路有些复杂，但是它的工作原理却很好理解。我们知道，从存储

器里取出一个二进制数（并将地址计数器加一），需要三个步骤；而在第 10 章里，把一个二进制数装载到寄存器 RA 中，或者用另一个数与 RA 中的数相加，分别需要两个步骤。现在，取数和做加法已经被合二为一，这就是为什么循环移位寄存器 RR 现在有 5 个输出 t_0、t_1、t_2、t_3 和 t_4 的原因。

两个功能的合并意味着需要重新设计一个新的逻辑电路来产生各种输出，这就是图中所示的“译码器”。所幸的是，这对现在的你来说并不困难，是不是？

在这台新机器上操作，和以前并无二致，开关 K 依然是我们使用的主要道具。每次当“1”从 t_0 循环移位到 t_3 时（换句话说，t_0～t_3 阶段），将完成从存储器里取数到寄存器 DR，并将地址自动加一的功能。这个阶段的功能永远是固定的，与 $K_{装载}$ 和 $K_{相加}$ 无关，不受它们的影响，这一点在设计译码器时需要注意。

在 t_3 和 t_4 阶段，执行的功能要取决于 $K_{装载}$ 和 $K_{相加}$ 的状态。如果 $K_{装载}=1$ 而 $K_{相加}=0$，则将 DR 中的数装载到寄存器 RA 中；如果 $K_{装载}=0$ 而 $K_{相加}=1$，则将 DR 中的数与 RA 中的数相加，并将结果保存到 RA 中。

所以很显然，要想把一大堆数加起来，只需要在存储器里准备好它们，并将地址计数器 AC 清零，然后坐下来，以 5 次为单位，不停地按动开关 K 即可。

合上 $K_{装载}$，断开 $K_{相加}$，“啪啪啪啪啪”地将 K 按 5 次；再断开 $K_{装载}$，合上 $K_{相加}$，接着啪啪……这个过程中有自动的成分。毕竟，我们只是用手动动开关，其他事情都由机器做了。当然，在这个过程中，唯一麻烦的就是 $K_{装载}$ 和 $K_{相加}$ 这两个开关，你要在适当的时候切换一下它们的状态。虱子多了不怕咬，在需要一堆开关才能做加法的时候，多它们两个不多，少它们两个不少；现在，一切都自动化了，这两个家伙就显得碍手碍脚——要是把它们省掉，那该多好哇。

很清楚，要想进一步自动化，这两个家伙必须下岗。不过，你也不要小看了这两个家伙，关键人物通常都在幕后，而且表面上都很普通。在图 11.20 里，开关 K 可能是最风光的，这台机器必须不停地按动它才能动起来。问题在于，同样是按 K，这台机器却会根据 $K_{装载}$ 和 $K_{相加}$ 的开合状态而做不同的事情。

换句话说，这台机器是按指令行事的。合上 $K_{装载}$，表示下达的是装载指令；合上 $K_{相加}$，下达的就是相加指令。同样是按动 K，指令不同，工作过程也不一样。

说了这么多，现在的问题是，我们就是不喜欢这两个开关，如何才能将它们撵走呢？办法是有的，这个问题的答案来自于一个叫诺依曼的人。

约翰·冯·诺依曼 1903 年 12 月 28 日生于匈牙利的布达佩斯，父亲是一个银行家。正是因为家里不差钱，据说这个“冯”姓是花钱买了一个爵位得来的。在欧洲一些国家，“冯”是个贵族的姓，比如历史上有名的铁血宰相冯·俾斯麦。

富家出人才，也出纨绔子弟；穷家出人才，也出无赖。诺依曼大抵是很懂事的，从小敏而好学，先后在苏黎世高等技术学校和布达佩斯大学攻读化学和数学，1927 年在柏林大学任教，1930 年赴美国普林斯顿大学担任讲师和教授。

从第二次世界大战的时候开始，由于军事上的需要，他还参与了计算机方面的研究工作，主要侧重于计算机的组织形式和体系结构。战后，他开始研究自动机理论，并对自动机和人脑思维过程的特点进行了比较。作为成果，他写了一本讲稿，名字叫《计算机与人脑》。可惜的是，还没有写完，他就于1957年去世了，所以这也是他最后一本著作。

目前为止，我们的存储器里全都是一些等待被加起来的数字。但是冯·诺依曼认为，存储器里不但要有这些纯粹的数字，还应当有一些指示如何加工这些数字的指令。在《计算机与人脑》这本书里，他写道："一条指令，在物理意义上和一个数是相同的。"换句话说，它们躺在存储器里，很像普通的二进制数，但实际上不是。就好比你大喊一声"狗！"，有时候，你指的是那种会汪汪叫的动物，但在另外一些场合，这会招来一顿拳脚。

如图11.21所示，所有的指令都以一个操作码开始，它指示出该指令的功能。比如，可以用10001表示"装载"，用10010表示"相加"。

地址	存储内容	具体含义
0000	10001	装载（RA←下一个存储单元里的数）
0001	01010	数（10）
0010	10010	相加（RA←RA+下一个存储单元里的数）
0011	00101	数（5）
0100	10010	相加（RA←RA+下一个存储单元里的数）
0101	00111	数（7）
0110	10010	相加（RA←RA+下一个存储单元里的数）
0111	00010	数（2）
1000	10010	相加（RA←RA+下一个存储单元里的数）
1001	00110	数（6）
1010	未使用	
⋮	⋮	
1111	未使用	

图11.21　保存了一些指令后的存储器布局

除此之外，操作码还隐含了一些别的意思。比如装载指令，往哪里装载呢？当然不是翻斗车。而且，装载谁呢？这个数字在哪里？所以，操作码10001还意味着，被装载的数位于下一个存储单元里，目标是寄存器RA。

相应的，相加指令的操作码10010则隐含了更多的意思。首先，它指出，第一个相加的数字位于寄存器RA；第二个相加的数字位于下一个存储单元；最后，相加的结果还要保存在寄存器RA里。

因此，一条完整的指令总是以操作码开始，后面跟着操作数。不得不说的是，有的指令可能不需要任何操作数，只有操作码就行了；而有的指令则很复杂，可能需要不止一个操作数。

在做出了这种安排之后，剩下的问题就是如何译出这些操作码，并用来代替

那两个开关 $K_{装载}$和 $K_{相加}$。这好像不难，考虑到逻辑电路的输出不是 0 就是 1，等效于开关 $K_{装载}$和 $K_{相加}$的闭合与断开，那么，使用我们非常熟悉的译码器来模拟传统的开关，应该是很不错的。

接下来，我们该如何让机器取出这些指令，并执行它们呢？我不想再跟你绕弯子，搞循循善诱、逐步推导，然后得出结论那一套。这本书读到这里，该有的基础你都有了，所以我直接给出结果，如图 11.22 所示。

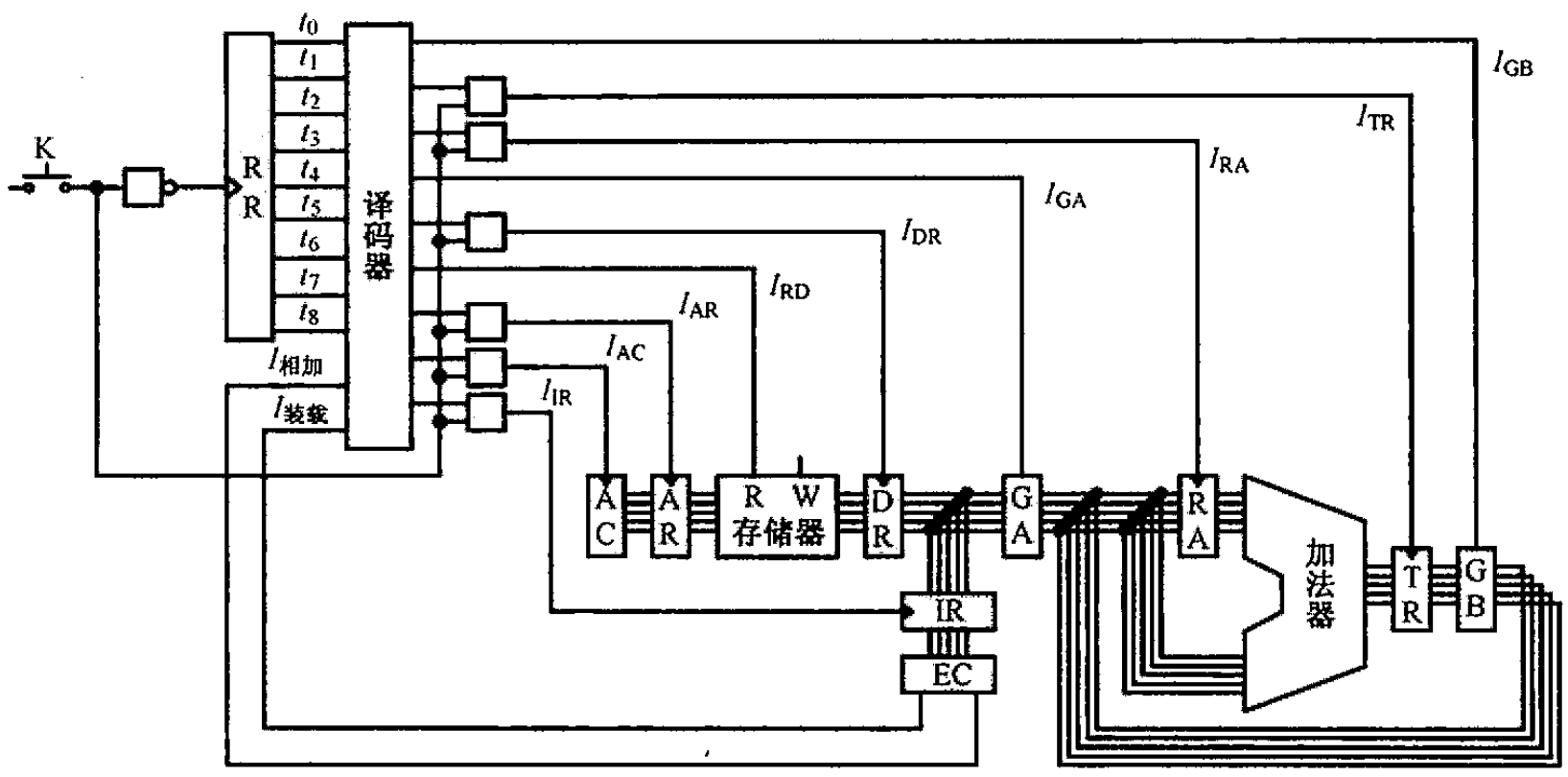


图 11.22　用一只开关就可完成将所有的数从存储器里依次取出并逐个相加的过程

假设这台机器刚刚通电启动。随着开关 K 的按动，在 t_0～t_2阶段，第一个存储单元（地址 0000）里的操作码被取出，地址计数器 AC 自动加一。这个阶段，称为取指令阶段。

现在，第一条指令的操作码已经位于寄存器 DR 中。在 t_3阶段，I_{IR}产生一个上升沿，使得寄存器 IR 将该操作码保存起来。IR 是一个普通的寄存器，但专门用来临时保存指令，所以称为指令寄存器。

IR 的输出直接通向译码电路 EC，EC 的任务是翻译当前指令，看它到底想做什么。当它的输入为 10001 时，$I_{装载}=1$，$I_{相加}=0$；相反，如果输入为 10010，则 $I_{装载}=0$，$I_{相加}=1$。而对于其余任何输入，$I_{装载}$和 $I_{相加}$都为 0。所以，t_3阶段称为指令译码阶段。

注意，$I_{装载}$和 $I_{相加}$对 t_0～t_3阶段没有影响，不管它们俩输入的是什么，机器所执行的都是取指令和翻译指令。

因为第一条指令是装载指令，所以 $I_{装载}=1$ 而 $I_{相加}=0$，于是从 t_4～t_8阶段，将依次执行下面的任务：从下一个存储单元里取数、地址计数器 AC 加一、把取出来的数装载到寄存器 RA 中。

至此，第一条指令执行完毕，循环移位寄存器 RR 已经经历了一次完整的循环移位。

在第二个 t_0～t_3 阶段，将取出第二条指令（相加指令）并进行译码，使得 $I_{装载}=0$，$I_{相加}=1$。于是，在第二个 t_4～t_8 阶段，将再次取数，并与 RA 中的数相加（结果依然返回 RA 中）。

基本上，这台机器的工作过程就是这样。你可以继续按动开关 K，直到所有的指令都执行完毕。

11.5 全自动计算

在上面的例子里，我们有 5 个数要相加：

$$10+5+7+2+6$$

为此，需要编制 5 条指令。为了执行每条指令，你都得按动 9 次开关 K，所以你唯一的工作就是坐下来，不停地按动那个开关。很容易计算，你总共需要不停地反复按 5×9=45 次。要是数字更多，那就只能恭喜你受累了。

当然，能够达到这一步，已经很先进、很了不起了。但是我敢保证，要是你在单位里也造一台这样的机器，要不了几天，这负责按开关的小子就会请你吃饭，打听这种枯燥的活儿能不能也交给机器来干。这不奇怪，人类喜欢偷懒，总想省事儿省力气。再说，人类有很强的机动性，但不如机器精确，这啪啪地按开关，保不准多按一次少按一次，题就算错了。

也许你不曾想到，按动开关只是用来发出一个由低到高、再到低的脉冲。如果连续按动，就相当于一个人肉振荡器。那么，为什么不到电子工程师那里拿一个振荡器来用呢？

我们没理由不这样做，但是这同时也带来了一个问题。当我们用手工来产生类似于振荡器的脉冲时，一切都是可以控制的，主要是脉冲次数。如果使用振荡器，你将不知道机器在工作的时候已经经历了多少个脉冲。当所有的数都已经加完之后，振荡器必须恰到好处地停下来，但遗憾的是这些无机物不像你一样有脑子。在这种糟糕的情况下，这台机器将持续计算，直到地址计数器 AC 计数到最大，然后又接着从 0000 开始重新计数。当你意识到应当撤除振荡器的时候，除了一个错误的结果之外，你什么也得不到。

为了获得使用振荡器带来的好处而又能使它处于可控的状态，需要做几个方面的改进工作。

首先，必须为这台机器增加一个新的指令，即停机指令，比如 11111，并把它放到其他所有指令的后面。不像我们已知的其他指令，它只有操作码而没有操作数，实际上也不需要。

其次，重新设计译码电路 EC，使它除了可以译出 $I_{装载}$和 $I_{相加}$之外，还能译出“停机”，即 $I_{停机}$。

最后，也是最重要的部分，重新设计这台机器的控制器，如图 11.23 所示。

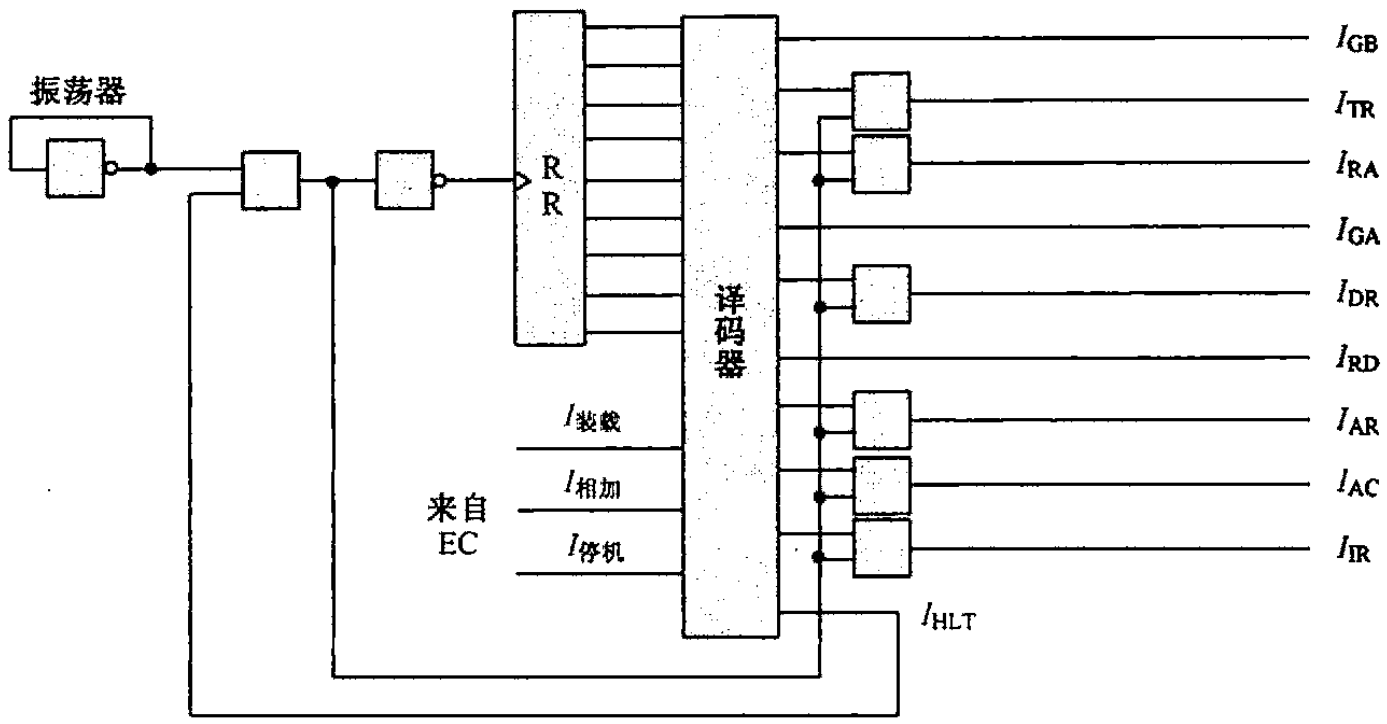


图 11.23　重新设计的控制器

显然，我们重新设计了指令译码电路 EC，使它可以译出 $I_{停机}$。平时，$I_{HLT}=1$，振荡器的脉冲可以顺利通过与门。一旦执行了停机指令，则 $I_{停机}=1$，这直接导致 $I_{HLT}=0$，于是不再有振荡器脉冲到达控制器，控制器停止工作，整个机器也就休息。这是一个富有喜剧色彩的动作，控制器给自己施了定身法，让自己僵在那里动弹不得。

计算机要想可靠地工作，指令的正确性至关重要。在存储器里，指令和普通的二进制数没有区别，但它们却有着独特的含义和用途。指令的数量是有限的，所以并非任意一个二进制比特串都代表一条指令。比如，1000100100 可能是某台计算机的一条指令，但 1000011110 则可能不是。如果计算机执行了并非指令的“指令”，指令译码器将不能输出正确的信号给控制器，整个计算机也就瘫痪了。

所有指令在存储器中的布置都是精心的，绝对不能错乱。当一条指令执行完后，紧接着，控制器取出的应当是另一条指令。不过，由于各种不同的原因，存储器中本应该是一条指令的地方恰恰是一个普通的二进制数，而非一条指令。计算机执行的非计算机指令称为非法指令。在早期，执行一条非法指令会引起我们通常所说的“死机”现象，因为控制器不知道该如何发出一系列控制信号来协调各个部件的动作，而现代的计算机则会自动从这种不正常的状态中恢复过来。